

chapter 10

List Interface

The **List interface** (`java.util.List`) is an ordered collection (also known as a *sequence*). It maintains the insertion order of elements, permits duplicate values, and allows positional access and insertion via zero-based indexing.

Java

```
import java.util.*;

public class ListExample {

    public static void main(String[] args) {

        // Using List interface as reference type
        List<String> fruits = new ArrayList<>();

        fruits.add("Apple");

        fruits.add("Banana");

        fruits.add("Apple"); // Duplicates allowed


        System.out.println("First element: " + fruits.get(0)); // Output: Apple

        System.out.println("Full List: " + fruits);           // Output: [Apple, Banana, Apple]

    }

}
```

2. ArrayList

An **ArrayList** is a dynamic resizable array implementation of the List interface. Unlike standard arrays, its capacity grows automatically when elements are added. It provides fast positional access ($O(1)$) but slower insertions or deletions in the middle of the list ($O(n)$) due to element shifting.

Java

```
import java.util.ArrayList;

public class ArrayListExample {

    public static void main(String[] args) {

        ArrayList<Integer> numbers = new ArrayList<>();

        numbers.add(10);

        numbers.add(20);

        numbers.add(30);


        // Modifying an element

        numbers.set(1, 25); // Replaces 20 with 25


        // Removing an element

        numbers.remove(0); // Removes element at index 0 (10)


        System.out.println(numbers); // Output: [25, 30]

    }

}
```

3. Stack

The **Stack** class extends Vector and represents a **LIFO (Last-In-First-Out)** data structure. Elements are pushed onto the top of the stack and popped off in reverse order. Primary methods include push(), pop(), peek(), and isEmpty().

Note: In modern Java code, ArrayDeque is generally preferred over Stack for single-threaded LIFO operations because Stack inherits synchronization overhead from Vector.

Java

```
import java.util.Stack;
```

```
public class StackExample {  
    public static void main(String[] args) {  
        Stack<String> bookStack = new Stack<>();  
  
        // Push elements onto the stack  
        bookStack.push("Book 1");  
        bookStack.push("Book 2");  
        bookStack.push("Book 3");  
  
        // View top element without removing  
        System.out.println("Top element: " + bookStack.peek()); // Output: Book 3  
  
        // Pop top element  
        System.out.println("Removed: " + bookStack.pop()); // Output: Book 3  
        System.out.println("Remaining Stack: " + bookStack); // Output: [Book 1, Book 2]  
    }  
}
```